

WE G(A)T NEWS

Digitale Schülerzeitung am Gymnasium Am Turmhof / Altenforst.

Diese Webseite ist eine statische Blog-Seite für Artikel der Schülerzeitung. Das Projekt ist bewusst einfach gehalten: Artikel werden als Markdown-Dateien gepflegt, Bilder liegen als WebP-Dateien im Repository, Netlify baut daraus automatisch die öffentliche Webseite.

Live-Seite: <https://wegatnews.de>

Aktueller Status

Das Projekt befindet sich noch in der Aufbauphase. Einige technische Schutzmechanismen sind deshalb noch bewusst einfach gehalten oder noch nicht eingerichtet.

Aktuell gilt:

- Developer können Artikel veröffentlichen.
- Es gibt momentan noch keine vollständige Branch-Protection auf `main`.
- Es gibt momentan noch keine zentrale automatische Nummernvergabe für Artikel.
- Die Tools unter `/tools` sind absichtlich öffentlich erreichbar.
- Die Seite wird automatisch über Netlify gebaut und deployed.
- Website-Statistiken laufen über Umami.
- Die Domain `wegatnews.de` wird über Strato verwaltet.

Das Ziel ist erstmal: Die Redaktion soll Artikel möglichst einfach schreiben, vorbereiten, prüfen und veröffentlichen können.

Tech Stack

Die Webseite basiert auf:

- **Next.js** mit **Pages Router**
- **React**
- **JavaScript**
- **CSS** in mehreren globalen Stylesheets
- **Markdown** für Artikel
- **gray-matter** zum Lesen des Frontmatters aus Markdown-Dateien
- **remark** und **remark-html** zum Umwandeln von Markdown in HTML
- **Turndown** im Editor, um HTML/Inhalte aus dem Browser in Markdown umzuwandeln
- **next-sitemap** für Sitemap und `robots.txt`
- **feed** für den RSS-Feed
- **GitHub Actions** für den Artikel-Workflow
- **Netlify** für Hosting, Deploys und Deploy Previews
- **Umami Cloud** für Website-Statistiken

Grundidee der Seite

Die Seite ist kein klassisches CMS wie WordPress. Stattdessen liegt alles direkt im GitHub-Repository:

- Der Code der Webseite liegt in `pages`, `components`, `lib` und `styles`.
- Die Artikel liegen als Markdown-Dateien in `posts`.
- Die Artikelbilder liegen als WebP-Dateien in `public/article-images`.
- Neue Artikel werden zuerst in `neuer-post` abgelegt.
- Ein GitHub-Workflow verschiebt neue Artikel an die richtige Stelle und erstellt einen Pull Request.
- Netlify erzeugt für Pull Requests automatisch eine Vorschau.
- Nach dem Merge baut Netlify die echte Webseite neu und veröffentlicht sie auf `wegatnews.de`.

Dadurch kann die Redaktion Artikel prüfen, bevor sie live gehen.

Repository-Struktur

```
.
├── .github/workflows/      # GitHub Actions, z. B. Artikel-Preview-
Workflow
├── components/            # Wiederverwendbare React-Komponenten
├── lib/                   # Hilfsfunktionen, z. B. Artikel laden und
Markdown verarbeiten
├── neuer-post/            # Zwischenablage für neue Artikel vor dem
Workflow
├── pages/                 # Next.js Pages Router: Seiten und Routen
├── posts/                 # Veröffentlichte Artikel als Markdown-Dateien
├── public/                # Statische Dateien, Bilder, RSS usw.
│   └── article-images/    # Artikelbilder als .webp
├── scripts/               # Hilfsskripte, z. B. RSS-Generierung
├── styles/                # Globale CSS-Dateien
├── next-sitemap.config.js  # Sitemap-/robots.txt-Konfiguration
├── next.config.js         # Next.js-Konfiguration
├── package.json           # npm-Skripte und Abhängigkeiten
└── README.md              # Diese Erklärung
```

Artikel-System

Ein Artikel besteht immer aus zwei Dateien:

```
posts/31.md
public/article-images/31.webp
```

Die Zahl ist die Artikelnummer. Markdown-Datei und Bilddatei müssen dieselbe Nummer haben.

Beispiel:

```
31.md
31.webp
```

Die Markdown-Datei enthält oben ein Frontmatter mit Metadaten und darunter den eigentlichen Artikeltext.

Beispiel:

```
---
title: "Beispielartikel"
articleNumber: "31"
slug: "beispielartikel"
date: "2026-05-25"
author: "Name, Klasse"
image: "/article-images/31.webp"
image_source: "eigene Aufnahme"
preview: "Kurzer Vorschautext für die Startseite."
tags: ["informativ"]
---

Hier beginnt der Artikeltext.
```

Wie Artikel technisch geladen werden

Die Datei `lib/posts.js` liest die Markdown-Dateien aus dem Ordner `posts`.

Dabei passiert grob Folgendes:

1. Alle `.md`-Dateien aus `posts` werden gelesen.
2. `gray-matter` trennt Frontmatter und Artikelinhalt.
3. Aus dem Frontmatter werden Daten wie Titel, Datum, Autor, Bild und Vorschau gelesen.
4. Wenn ein `slug` vorhanden ist, wird dieser für die URL genutzt.
5. Wenn kein `slug` vorhanden ist, wird aus dem Titel automatisch ein Slug erzeugt.
6. Artikel werden nach Datum sortiert.
7. Für die Artikelseite wird der Markdown-Inhalt mit `remark` und `remark-html` in HTML umgewandelt.

Die URLs der Artikel sehen ungefähr so aus:

```
/blog/beispielartikel/
```

Wichtig: Wenn sich der Slug eines Artikels nachträglich ändert, ändert sich auch die URL des Artikels.

Seiten und Routing

Das Projekt nutzt den Next.js Pages Router. Das bedeutet: Dateien im Ordner `pages` werden automatisch zu Routen.

Beispiele:

```
pages/index.js           -> /
pages/blog/[slug].js     -> /blog/<slug>
pages/tools/index.js      -> /tools
pages/tools/editor.js     -> /tools/editor
pages/tools/converter.js  -> /tools/converter
```

Die Hauptnavigation wird im Layout definiert und führt aktuell unter anderem zu:

- Informativ
- Meinung
- Umfragen
- Mach mit

Layout und Styling

Das globale Layout liegt in `components/layout/SiteLayout.js`.

Dort werden unter anderem eingebunden:

- Header
- Navigation
- Mobile Navigation
- Footer
- Links zu Datenschutz, Impressum und Cookies
- Umami-Tracking-Script

Die Styles liegen in `styles` und werden global in `pages/_app.js` importiert.

Beispiele:

```
import "../styles/globals.css";
import "../styles/layout.css";
import "../styles/components.css";
import "../styles/post.css";
import "../styles/tools.css";
```

Tools unter `/tools`

Unter `https://wegatnews.de/tools` gibt es Hilfswerkzeuge für die Redaktion.

Diese Tools sind absichtlich öffentlich erreichbar. Sie speichern keine Artikel auf dem Server, sondern laufen im Browser der Nutzerinnen und Nutzer.

Editor

Der Editor hilft dabei, eine fertige Markdown-Datei für einen Artikel zu erzeugen.

Er kann:

- Titel eintragen
- Artikelnummer eintragen
- Datum setzen
- Autor eintragen
- Bildquelle eintragen
- Vorschau eintragen
- Tags eintragen
- Artikeltext aus einem Editorfeld übernehmen
- Inhalte beim Einfügen bereinigen, z. B. aus Word
- Markdown erzeugen
- die Datei als `NUMMER.md` herunterladen

Der Editor prüft unter anderem:

- Titel darf nicht zu lang sein
- Vorschau darf nicht zu lang sein
- Artikelnummer darf nur aus Zahlen bestehen
- Titel, Vorschau, Autor und Text müssen vorhanden sein

Converter

Der Converter bereitet Artikelbilder vor.

Er erzeugt aus einem hochgeladenen Bild eine passende WebP-Datei für die Webseite.

Ziel:

`NUMMER.webp`

Die Datei wird danach zusammen mit der Markdown-Datei in `neuer-post` hochgeladen.

Artikel veröffentlichen

Der normale technische Ablauf für einen neuen Artikel ist:

1. Artikel im Editor vorbereiten und als `NUMMER.md` herunterladen.
2. Bild im Converter vorbereiten und als `NUMMER.webp` herunterladen.
3. Beide Dateien in den Ordner `neuer-post` hochladen.
4. In GitHub den Workflow **Neuer Post Preview** starten.
5. Der Workflow erstellt einen Pull Request.

6. Netlify erstellt automatisch einen Deploy Preview für diesen Pull Request.
7. Die Vorschau wird geprüft, möglichst auch auf dem Handy.
8. Wenn alles passt, wird der Pull Request merged oder über den vorgesehenen Publish-Weg veröffentlicht.
9. Netlify baut danach automatisch die Live-Seite neu.
10. Der Artikel ist kurz danach auf `wegatnews.de` sichtbar.

GitHub-Workflow für neue Artikel

Der Workflow liegt unter:

```
.github/workflows/new-post-preview.yml
```

Er wird manuell über GitHub Actions gestartet.

Der Workflow macht aktuell grob Folgendes:

1. Repository auschecken
2. Node.js installieren
3. Dependencies mit `npm ci` installieren
4. Markdown-Datei in `neuer-post` suchen
5. Artikelnummer aus dem Dateinamen erkennen
6. passendes Bild `NUMMER.webp` suchen
7. prüfen, ob Artikel oder Bild schon existieren
8. Markdown nach `posts/NUMMER.md` kopieren
9. Bild nach `public/article-images/NUMMER.webp` kopieren
10. Build mit `npm run build` testen
11. `neuer-post` leeren
12. Branch `post/NUMMER` erstellen
13. Änderungen committen und pushen
14. Pull Request gegen `main` erstellen

Der Workflow nutzt dafür einen GitHub Token bzw. ein Secret.

Build und lokale Entwicklung

Voraussetzungen

- Node.js
- npm
- Git

Im Workflow wird Node.js 22 verwendet. Lokal sollte eine aktuelle Node-Version genutzt werden.

Installation

```
npm install
```

oder sauber nach Lockfile:

```
npm ci
```

Entwicklungsserver starten

```
npm run dev
```

Danach ist die Seite lokal unter dieser Adresse erreichbar:

```
http://localhost:3000
```

Produktions-Build testen

```
npm run build
```

Der Build ist wichtig, weil Netlify die Seite ebenfalls baut. Wenn der Build lokal fehlschlägt, wird meistens auch der Deploy fehlschlagen.

Seite starten nach Build

```
npm run start
```

Linting

```
npm run lint
```

npm-Skripte

Die wichtigsten Skripte aus `package.json`:

```
{
  "dev": "next dev",
  "build": "next build",
  "start": "next start",
  "lint": "eslint",
  "postbuild": "next-sitemap && npm run generate-rss",
  "generate-rss": "node scripts/generate-rss.js"
}
```

Besonders wichtig ist `postbuild`:

Nach `next build` werden zusätzlich Sitemap und RSS-Feed erzeugt.

Sitemap, robots.txt und RSS

Die Seite erzeugt automatisch:

- Sitemap
- `robots.txt`
- RSS-Feed unter `/feed.xml`

Die Sitemap-Konfiguration liegt in:

```
next-sitemap.config.js
```

Der RSS-Feed wird erzeugt durch:

```
scripts/generate-rss.js
```

Der RSS-Feed nutzt die Artikel aus `posts` und erzeugt daraus eine Datei:

```
public/feed.xml
```

Hosting und Deployment

Die Webseite wird über Netlify gehostet.

Netlify ist mit dem GitHub-Repository verbunden.

Aktueller Ablauf:

- Änderungen auf `main` lösen einen Production Deploy aus.
- Pull Requests erzeugen Deploy Previews.
- Der Workflow für neue Artikel erstellt Pull Requests, damit Artikel vor dem Veröffentlichen geprüft werden können.
- Nach dem Merge baut Netlify die Live-Seite neu.

Die Domain `wegatnews.de` zeigt auf die Netlify-Seite.

Umami Analytics

Website-Statistiken laufen über Umami Cloud.

Das Tracking-Script ist im Site Layout eingebunden:


```
<script
  defer
  src="https://cloud.umami.is/script.js"
  data-website-id="2887ff34-c251-4b42-b710-7c2e064c633a"
></script>
```

Damit werden Seitenaufrufe gezählt.

Hinweis: Da das Script im Layout eingebunden ist, sollte geprüft werden, ob auch Netlify-Previews gezählt werden. Falls ja, können interne Vorschau-Aufrufe die Statistik verfälschen.

Aktuelle Sicherheits- und Organisationslage

Das Projekt ist noch im Aufbau. Deshalb ist die technische Organisation momentan bewusst einfach.

Aktuell:

- Developer können veröffentlichen.
- Es gibt noch keine vollständige Branch Protection.
- Die Artikelnummern werden nicht zentral automatisch vergeben.
- Die Tools sind öffentlich erreichbar.
- Der Veröffentlichungsprozess verlässt sich auf sorgfältiges Arbeiten der Technik-Redaktion.

Später mögliche Verbesserungen:

- Branch Protection für `main`
- verpflichtender Pull Request vor Merge
- verpflichtender erfolgreicher Build vor Merge
- automatische Prüfung von Artikelnummern
- automatische Prüfung von Pflichtfeldern wie Bildquelle
- klare Rollen: schreiben, prüfen, technisch veröffentlichen
- Schutz vor gleichzeitigen Uploads in `neuer-post`
- bessere Fehlermeldungen im Workflow

Häufige Fehler

Markdown-Datei fehlt

In `neuer-post` muss eine `.md`-Datei liegen.

Bild fehlt

Zu `31.md` muss auch `31.webp` existieren.

Nummern passen nicht zusammen

Falsch:

```
31.md  
32.webp
```

Richtig:

```
31.md  
31.webp
```

Artikel existiert schon

Wenn `posts/31.md` oder `public/article-images/31.webp` schon existiert, sollte eine andere freie Artikelnummer verwendet werden.

Build schlägt fehl

Dann liegt meistens ein Fehler in Markdown, Frontmatter, Code oder Dateipfad vor. Vor dem Veröffentlichen muss der Build wieder funktionieren.

Für neue Technik-Mitglieder

Du musst nicht alles sofort verstehen.

Der wichtigste Ablauf ist:

```
Markdown + Bild vorbereiten  
-> in neuer-post hochladen  
-> Workflow starten  
-> Netlify Preview prüfen  
-> Pull Request mergen  
-> Netlify veröffentlicht die Seite
```

Das System ist so gebaut, dass man ohne Terminal und ohne lokale Entwicklungsumgebung Artikel veröffentlichen kann. Für normale Artikel reicht GitHub im Browser.

Wer am Code arbeiten möchte, kann das Projekt lokal mit `npm run dev` starten und Änderungen testen.

Kurzfassung

WE G(A)T NEWS ist eine statische Next.js-Blogseite für eine Schülerzeitung.

Artikel werden nicht in einer Datenbank gespeichert, sondern als Markdown-Dateien im Repository. Bilder liegen als WebP-Dateien im `public`-Ordner. GitHub Actions bereiten neue Artikel als Pull Request vor. Netlify baut Vorschauen und veröffentlicht die Live-Seite. Umami zählt die Seitenaufrufe. Die Tools unter `/tools` helfen der Redaktion, Artikeldateien und Bilder korrekt vorzubereiten.